An Analysis of the XRP Ledger Consensus Protocol

Dejan Cabrilo[1], Odysseas Sofikitis[1,2], Dionysis Zindros[2]

[1] Common Prefix

[2] National Technical University of Athens

July 2, 2026

1 Introduction

The XRP Ledger (XRPL) is a Byzantine fault-tolerant replicated ledger whose validators maintain a shared chain to which ledgers are appended. Contrary to most quorum-based State Machine Replication (SMR) protocols, the validators do not share the same set of peers. Each validator holds its own set of trusted peers, called a *Unique Node List*, and only receives messages from the validators in this set.

XRPL validators repeatedly run rounds of consensus. Each round consists of three phases: (i) an Open phase that gathers candidate transactions, (ii) an Establish phase that converges on a common transaction set through avalanche voting, and (iii) an Accepted phase that builds and attests the resulting ledger. A fork-choice rule keeps validators on the same branch.

This document outlines the core consensus protocol, abstracting away engineering details and optimizations. The pseudocode is based on the reference implementation (xrpld) and mirrors its structure and function names so that each rule can be traced back to its source.

Section 2 discusses prior descriptions and analyses of the protocol. In Section 3, we define the adversarial and network model and the cryptographic assumptions under which we operate. We also define the data structures and network messages of the protocol that help us define the security properties we aim to explore in this work. Section 4 specifies the protocol; we present the validator state, the events, and the pseudocode for each phase. We examine both a single round of the protocol and the overall behavior in repeated rounds (e.g. fork choice and finality). Finally, we analyze the security of the protocol in Section 5.

2 Related work

The XRPL consensus protocol has been described and analyzed several times before. The first description was given by Schwartz *et al.* [1] in 2014. This was a high-level approach that included neither a formal description of the protocol nor formal security arguments, aside from some intuition for the structure of the UNLs. It claims that a 20% overlap of the UNLs is sufficient for safety at an 80% quorum size. The following year, the work of Armknecht *et al.* [2] offered an early analysis, including some quorum-intersection arguments. It contradicts the previous claim of 20% overlap, arguing that the actual overlap should be at least 40%. However, the work is still high level; it does not provide a formal description of the protocol, nor any definitions of the security properties, while the modeling is lacking in detail. For example, the quorum-intersection arguments do not consider the possibility of an adversarial validator equivocating and voting for two conflicting ledgers.

Later, in 2018, Chase and MacBrough [3] gave a more academic treatment of the protocol. They formalized the model of the validators, messages, adversarial behavior and provided some security properties based on *atomic broadcast*. The paper also outlines the protocol in pseudocode, giving a first look at implementation details such as the fork choice rule. The pseudocode provided closely matches the current implementation and uses similar terminology. Lastly, the paper provides a formal analysis for specific aspects of the protocol: (a) it formalizes the quorum-intersection argument and the UNL overlap needed for safety, disputing the claims of the previous works, and (b) it analyzes the liveness of the protocol under a 99% UNL overlap.

In 2020, there were two independent works that formalized the protocol. The first one, by Mauri *et al.* [4], provided a description of the protocol and its components, taking a deep dive into the different "consensus phases", and explicitly stating some properties, quorums, and assumptions. However, the

analysis is not complete. The paper describes the parameters of the protocol and their impact on safety and liveness, but does not provide formal proofs for the properties.

Finally, the work of Amores-Sesar *et al.* [5] is the first that provides a full description of the protocol, accompanied by a detailed pseudocode, depicting all the different phases of the protocol, including the fork choice rule. This work does not include any security analysis. Instead, it provides the parameters and assumptions under which concrete attacks—described in the paper—are possible.

In this work, we aim first to formalize the protocol and its components and identify the areas of interest for our analysis. We wish to fix the assumptions and simplify any moving components of the protocol, in order to analyze a version that can capture the core consensus mechanism. This will lead to a formal analysis of the protocol, understanding its ins and outs and identifying the parameter space for which the protocol is (in)secure.

3 Model & Definitions

3.1 Assumptions

Validators. We consider a static set of parties $\mathcal{V}$, which we will refer to as *validators*. Each validator is configured with a set of trusted peers, called their *Unique Node List* (UNL). A validator only receives messages from the validators in their UNL set. For every validator V_i we denote their UNL set by UNL_i. For all validators V_i, $\mathsf{UNL}_i \subseteq \mathcal{V}$ and $\bigcup_{V_i \in \mathcal{V}} \mathsf{UNL}_i = \mathcal{V}$. For brevity, we sometimes refer to the size of UNL_i as n_i. We will initially explore the protocol under the assumption of total overlap among the UNL sets. We will relax this assumption in later iterations of the analysis.

Adversarial model. We consider a *Byzantine*, *rushing* adversary $\mathcal{A}$ that can arbitrarily deviate from the protocol. For every UNL UNL_i, $\mathcal{A}$ can corrupt up to $t_i = \lceil \frac{|\mathsf{UNL}_i|}{5} \rceil - 1$ validators. The adversary can *adaptively* corrupt validators, i.e. she does not have to decide the corrupted parties beforehand, and can choose to corrupt a party during the protocol execution.

Network model. We assume a synchronous network model with a network delay parameter Δ. Any message sent by an honest party at time t is received by all honest parties[1] by $t + \Delta$. We assume that parties have perfectly synchronized clocks with no drift.

Cryptography. We assume unforgeable digital signatures and a PKI which includes the public keys of all the validators. All parties have access to the public keys, which uniquely identify the validators. We assume a collision-resistant hash function H.

3.2 Data structures & Messages

Notation. Given a sequence Y, we address it using $Y[i]$ to mean the i^{th} element (starting from 0). We use $|Y|$ to denote the length of Y. Negative indices address elements from the end, so $Y[-i]$ is the i^{th} element from the end, and $Y[-1]$ in particular is the last. We use $Y[i{:}j]$ to denote the subsequence of Y consisting of the elements indexed from i (inclusive) to j (exclusive). The notation $Y[i{:}]$ means the subsequence of Y from i onwards, while $Y[{:}j]$ means the subsequence of Y up to (but not including) j. The notation $\|$ denotes the concatenation of two strings. Given a tuple $L = (a, b, c)$, we use the dot notation to access its fields, so $L.a$ is a, $L.b$ is b, and $L.c$ is c. If a tuple does not include a field d then $L.d = \perp$. We assume that all fields are initialized to $\perp$. We implicitly cast boolean variables to integers, using the convention that $\mathsf{true} = 1$ and $\mathsf{false} = 0$. We use the $\perp$ value to denote both the absence of a value and the deletion of a key-value pair in a dictionary.

Parties in this protocol create and hold a *chain* C of *ledgers* L. We model the chain as a sequence of ledgers, $C = (L_0, L_1, \ldots, L_n)$, where L_0 is the genesis ledger and L_n is the latest ledger in the chain. We call the position of a ledger in the chain its *sequence number*, denoted by seq.

Ledger. A ledger is a tuple $L = (\mathsf{parent}, \mathsf{seq}, \mathsf{txs})$:

- parent is a pointer to the previous ledger in the chain; for the genesis ledger, $\mathsf{parent} = \perp$;
- $\mathsf{seq} \in \mathbb{N}$ is the sequence number of the ledger; $L.\mathsf{seq} = L.\mathsf{parent}.\mathsf{seq} + 1$, and the $\mathsf{genesis}$ ledger has $\mathsf{seq} = 0$;
- txs is a canonically ordered list of transactions;

[1] The message will be received by the honest parties that consider the sender in their UNL set.

We often use the term id to refer to a unique identifier for a ledger, which is computed as $H(L)$, where H is a collision-resistant hash function.

Aside from the parent of a ledger, we also define the ANCESTORS relation, as the transitive closure of the parent relation. By construction, every ledger has a unique ancestor at every sequence number. In the pseudocode, we will use the function ANCESTOROF(L, s) to denote the ancestor of L at sequence number s.

Proposal. A proposal is a tuple $\mathsf{Proposal} = (\mathsf{prev\_ledger}, \mathsf{txs}, \mathsf{ver}, \mathsf{pk}_i, \sigma)$:

- $\mathsf{prev\_ledger}$ is the ledger the proposer extends;
- txs is the advocated transaction set;
- $\mathsf{ver} \in \mathbb{N}$ is a revision number;
- pk_i is the public key of the proposer V_i;
- σ is the signature of the proposal by V_i.

The special revision number $\mathsf{ver} = \mathsf{VER\_LEAVE}$ signals a bow-out. Proposals are signed messages sent by a validator to its peers during a consensus round.

Validation. A validation is a tuple $\mathsf{Validation} = (\mathsf{ledger}, \mathsf{pk}_i, \mathsf{full}, \sigma)$:

- ledger is the ledger that the validator validated;
- pk_i is the public key of the validator that issued the validation;
- $\mathsf{full} \in \{\mathsf{true}, \mathsf{false}\}$ is a flag indicating if the validation is full or partial;
- σ is the signature on the validation by V_i.

The flag full is set iff the validation was issued while proposing: only full validations count towards quorum, while both full and partial validations count towards the fork choice rule.

For brevity, during the protocol description, we will omit the signatures and their verification. We will assume that all messages are signed by the validator with their public key, and validators verify the signatures of the messages they receive. We also assume that all messages received by a validator are from their trusted validators and we omit the identity check.

3.3 Security properties

The goal of this protocol is to achieve State Machine Replication (SMR). Towards this end, we state the usual properties of SMR protocols. First, we define the concept of an "irrevocably finalized" ledger, i.e. a ledger we are sure will never be replaced by another ledger of the same sequence number.

Definition 1 (Fully validated ledger)**.** A ledger L is called *fully validated* in the view of a validator V_i, if V_i has received $n_i - t_i$ full validations for L.

The chain C of a validator V_i consists of a prefix of fully validated ledgers and possibly a suffix of ledgers that are not yet fully validated. We call a set of $n_i - t_i$ full validations on a ledger L a *quorum* for L. That is, a fully validated ledger has a quorum in the view of the validator. A chain is called *fully validated* if all its ledgers are fully validated.

Definition 2 (Safety)**.** Consider two honest validators V_i and V_j and the fully validated chains C_i and C_j of V_i and V_j, respectively. Then, either $C_i \preceq C_j$ or $C_j \preceq C_i$.

The fully validated chains of two honest validators are always a prefix of each other.

Definition 3 (Liveness (u))**.** Consider an honest transaction tx that is delivered to every honest validator at time t. Then, tx is included in the fully validated chain of every honest validator by $t + u$.

Honest transactions are always eventually included in the fully validated chain of every honest validator.

Definition 4 (Security)**.** A protocol is *secure* if it satisfies the *safety* and *liveness* properties.

4 Construction

We now present the construction of the XRP Ledger consensus protocol. We start by describing the flow of the protocol at a high level. Then, we get into the details, first describing the validator state, constants and the events that make the protocol run. We then describe the three phases of a consensus round, accompanied by the respective pseudocode. Finally, we describe the fork-choice rule and the finality mechanism.

4.1 Overview

The protocol consists of consecutive "consensus" rounds. Each round is a loop, which is triggered by a 1 Hz timer. At the start of the loop, the validator checks if they are extending the preferred ledger. If not, they bow out—i.e. they inform their peers that they are under re-org—and restart the loop on the preferred ledger. If they are extending the preferred ledger, they enter the Open phase, where they gather transactions and create a first proposal. Afterwards, they move on to the Establish phase. The Establish phase consists of an *avalanche* voting process. Validators share proposals on transaction sets, and vote in favor of or against each transaction. When enough validators agree on a transaction set or too much time has passed, they continue to the Accepted phase. There, the validator builds the new ledger, extending the old one, and shares it with their peers along with a Validation vote. A quorum of validations is needed to finalize the ledger. After the validator broadcasts their vote, a new consensus round is started using the new ledger.

4.2 Protocol state

4.2.1 Validator state

The validator keeps track of the ledger tree that is being built, the proposals and validations of peers, and its own local position during a consensus round. These are shown in Algorithm 1. Specifically, a validator keeps track of the following:

- Their trusted validators set UNL.
- The genesis ledger genesis.
- The ledger they are currently extending prev_ledger. Initialized to the genesis ledger.
- The ledger tree ledgers.
- The most recent fully validated ledger validated_ledger. Initialized to the genesis ledger.
- The highest sequence for which they have issued a full validation on a ledger, last_issued_seq. Initialized to 0, the sequence number of the genesis ledger.
- The current phase of the round phase.
- Whether the validator is proposing during this round proposing. Initialized to true. Changes to false when the validator bows out.
- The duration of the previous Establish phase prev_establish_time. Initialized to a default duration, ESTABLISH_INIT.
- The number of validators that sent a proposal during the last Establish phase, prev_proposers.
- The transaction set we are proposing this round txs.
- The version of the proposal we are sending this round prop_ver. Initialized to 0. Incremented by 1 for each new proposal we send.
- The latest validation issued by each validator last_validation.
- A mapping from (validator, ledger) pairs to validation votes, validations.
- The latest proposal sent by each validator peer_positions.
- The time passed since the last change of peers' positions peer_unchanged.

- Disputes opened for transactions during the avalanche voting phase, disputes.
- The peers that bowed out this round dead_nodes.

Algorithm 1 Validator state. Variables defined here are considered global and accessible from all validator functions.

```
pk                                          ▷ Public key of the validator
UNL                                         ▷ Trusted validators set
genesis                                     ▷ Root of the ledger tree
prev_ledger ← genesis                       ▷ Initialize ledger to genesis
ledgers ← {genesis}
validated_ledger ← genesis                  ▷ Most recent fully validated ledger
last_issued_seq ← 0                         ▷ Most recent sequence number issued
phase ← ⊥                                   ▷ Current phase of the round
proposing ← true
round_start ← 0                             ▷ Beginning of consensus round
establish_start ← 0                         ▷ Beginning of ESTABLISH phase
prev_establish_time ← ESTABLISH_INIT        ▷ Previous ESTABLISH duration
prev_proposers ← 0                          ▷ Number of proposers in the previous ESTABLISH phase
txs ← ∅                                     ▷ Transaction set proposed this round
prop_ver ← 0                                ▷ Proposal version
last_validation ← ∅                         ▷ Validator ↦ latest Validation
validations ← ∅                             ▷ (Validator, ledger) ↦ Validation
peer_positions ← ∅                          ▷ Validator ↦ latest Proposal
peer_unchanged ← 0                          ▷ Counts ticks since last change of peers' positions
disputes ← ∅                                ▷ Disputed transactions under avalanche voting
dead_nodes ← ∅                              ▷ Validators that bowed out this round
```

4.2.2 Validator events

The validator is synchronized to a 1 Hz timer. Whenever the timer fires, the ON_TICK() path is executed. Asides from this, the validator asynchronously receives and handles proposals and validations from their trusted peers.

Event	Trigger	Handler	Effect
ON_TICK()	every 1 sec	Algorithm 2	re-org onto preferred ledger if needed, then act on the current phase
ON_PROPOSAL(p)	receival of a Proposal	Algorithm 5	record the peer's latest tx set, refresh disputes, handle bow-outs
ON_VALIDATION(v)	receival of a Validation	Algorithm 8	grow the ledger tree, check quorum

Table 1: Validator events.

4.2.3 Constants

Throughout the protocol, some heuristic timers are used. These are either a minimum duration a validator needs to wait, or a maximum duration that a validator can spend in a phase. These are shown in Table 2.

Along with the formalization of the protocol, the goal is to translate these heuristic timers into concrete network delays and express them with respect to Δ. This will make the protocol easier to analyze and more robust to different network assumptions. The analysis will highlight the number of network round-trips needed for the validator in each case.

Constant	Value	Used in
MIN_OPEN	2 s	minimum duration of OPEN
ESTABLISH_WAIT	1.95 s	ESTABLISH wait
ESTABLISH_INIT	15 s	prev_establish_time bootstrap
ESTABLISH_MIN	5 s	floor of prev_establish_time in converge_pct
CONSENSUS_PCT	80	tx-set agreement threshold
AV_MIN_DUR	2	ticks in an avalanche state before it advances
AV_STALLED_DUR	4	ticks before a dispute is considered stalled
CONSENSUS_FACTOR	10	expiry deadline multiplier
CONSENSUS_MAX	120 s	cap of the expiry deadline
VER_LEAVE	`0xffffffff`	bow-out sentinel revision

Table 2: Protocol constants. The per-state avalanche thresholds AVALANCHE_CUTOFFS are given in Table 3.

4.3 The protocol

The protocol consists of consecutive consensus rounds. Each round builds a ledger on top of the previous one and passes through three phases: OPEN, ESTABLISH, and ACCEPTED. The round is driven by a timer which fires every some time interval. On every tick the validator first checks that it is still on the right branch (Section 4.4) and then acts according to the current phase.

Open phase

Initialization. STARTROUND begins a round on a parent ledger. It clears all per-round state and enters OPEN. Only two values carry over from the last completed round: the duration of the previous ESTABLISH phase prev_establish_time and the number of peers that proposed in it prev_proposers. These are used to pace the new round according to the previous round's duration and participation.

Open phase. In OPEN the validator collects transactions in their mempool. On each tick SHOULDCLOSELEDGER decides whether to close. It does so once more than half of last round's proposers have moved on to the new round, or once enough time has passed. CLOSELEDGER then fixes the transaction set txs, broadcasts a first Proposal (if proposing), opens a dispute for every transaction it disagrees on with a known peer, and enters ESTABLISH.

Establish phase

Overview. ESTABLISH is the main phase of the protocol where validators will try to converge on a common transaction set to be included in the ledger (Algorithm 3). Specifically, after everybody has broadcasted their proposals, the avalanche voting process is started, where each validator will change their transaction set based on the votes of their peers. Whenever a validator changes their transaction set, they broadcast a new proposal, with an incremented version number prop_ver. If the conditions in HAVECONSENSUS are met, the validator will move on to the ACCEPTED phase.

Avalanche voting. Every transaction the validator and a peer disagree on becomes a dispute, resolved by an avalanche vote (Algorithm 4). A dispute is a DisputedTx tuple (tx, vote, votes, avalanche_state, avalanche_counter):

- the disputed transaction tx,
- our current vote vote on whether to include it,
- a map votes from each peer to its vote on tx,
- the current avalanche state avalanche_state,
- avalanche_counter, the number of ticks spent in that state.

Algorithm 2 Round bootstrap, OPEN phase and closing the ledger

```
1: function startRound(parent, prop)
2:     phase ← OPEN
3:     proposing ← prop
4:     prev_ledger ← parent
5:     round_start ← now()
6:     peer_positions ← ∅
7:     disputes ← ∅
8:     dead_nodes ← ∅
9: end function

10: upon tick at local time now                                   ▷ The 1 Hz timer
11:     L' ← getPrevLedger()
12:     if L' ≠ prev_ledger then                                  ▷ Not extending preferred ledger
13:         if phase = ESTABLISH then                             ▷ Bow-out
14:             broadcast(Proposal(prev_ledger, txs, VER_LEAVE, pk))
15:         end if
16:         startRound(L', false)                                 ▷ Restart without proposing
17:     else if phase = OPEN ∧ shouldCloseLedger() then
18:         closeLedger()
19:     else if phase = ESTABLISH then
20:         phaseEstablish()
21:     end if
22: end upon

23: function shouldCloseLedger()
24:     c ← |{p ∈ peer_positions.values() : p.prev_ledger = prev_ledger}|   ▷ Peers have already closed
25:     c' ← |{((pk_i, L), v) ∈ validations : L = prev_ledger ∧ v.full}|    ▷ Peers are done with previous round
26:     if (c + c') · 2 > prev_proposers then                     ▷ Enough peers to begin new round
27:         return true
28:     else if now() − round_start < MIN_OPEN then               ▷ Not enough time elapsed
29:         return false
30:     end if
31:     ▷ Don't wait less than the previous establish duration
32:     return now() − round_start ≥ prev_establish_time
33: end function

34: function closeLedger()
35:     txs ← getMempool()
36:     prop_ver ← 0
37:     phase ← ESTABLISH
38:     establish_start ← now()
39:     peer_unchanged ← 0
40:     if proposing then
41:         broadcast(Proposal(prev_ledger, txs, prop_ver, pk))
42:     end if
43:     for p ∈ peer_positions.values() do
44:         createDisputes(p.txs)
45:     end for
46: end function
```

Algorithm 3 The ESTABLISH phase.

```
1: function phaseEstablish()
2:     peer_unchanged ← peer_unchanged + 1                ▷ Set to zero on new proposal
3:     τ ← now() − establish_start                        ▷ Time spent in Establish
4:     if τ < ESTABLISH_WAIT then
5:         return                                         ▷ Wait for others to catch up
6:     end if
7:     ▷ Time spent in current establish as proportion of the previous establish
8:     converge_pct ← ⌊100 · τ / max(prev_establish_time, ESTABLISH_MIN)⌋
9:     updateOurPositions(converge_pct)                   ▷ Update proposal if needed
10:    if ¬haveConsensus(τ) then
11:        return
12:    end if
13:    doAccept()
14: end function

15: function updateOurPositions(converge_pct)
16:    changed ← false
17:    for d ∈ disputes do
18:        if updateVote(d, converge_pct) then            ▷ Algorithm 4
19:            if d.vote then
20:                txs ← txs ∪ {d.tx}                     ▷ Add to proposal
21:            else
22:                txs ← txs \ {d.tx}                     ▷ Remove from proposal
23:            end if
24:            changed ← true
25:        end if
26:    end for
27:    if changed ∧ proposing then                        ▷ Propose if our position changed
28:        prop_ver ← prop_ver + 1
29:        broadcast(Proposal(prev_ledger, txs, prop_ver, pk))
30:    end if
31: end function
```

createDisputes opens a dispute for each transaction in the symmetric difference between our set and a peer's, setting vote to whether tx is in our set, votes from the peer positions on hand, avalanche_state to Init, and avalanche_counter to 0. A dispute moves through the states Init, Mid, Late, and Stuck. The threshold a transaction must reach for the validator to vote for it rises with the state, from 50% up to 95% (Table 3). The duration of an Establish phase is measured as a percentage of the previous Establish phase's duration, converge_pct. A dispute advances to the next state once it has held its current state for at least AV_MIN_DUR ticks and converge_pct has reached the next threshold. A validator that is not proposing ignores the thresholds and goes along with the majority.

State	at_pct	threshold	next
Init	0	50	Mid
Mid	50	65	Late
Late	85	70	Stuck
Stuck	200	95	Stuck

Table 3: AVALANCHE_CUTOFFS: thresholds per avalanche state.

Algorithm 4 Per-transaction avalanche voting

```
1:  function createDisputes(txs′)                                   ▷ tx set received from peer
2:      T ← txs △ txs′                                              ▷ Symmetric difference
3:      for tx ∈ T do
4:          if ∃d ∈ disputes: d.tx = tx then
5:              continue
6:          end if
7:          d ← (tx, tx ∈ txs, {(p.pk, tx ∈ p.txs) : p ∈ peer_positions}, Init, 0)   ▷ Create new dispute
8:          disputes ← disputes ∪ {d}
9:          peer_unchanged ← 0
10:     end for
11: end function

12: function updateVote(d, converge_pct)                            ▷ Returns true iff d.vote flips
13:     nays ← |{vote ∈ d.votes.values(): ¬vote}|
14:     yays ← |{vote ∈ d.votes.values(): vote}|
15:     if (d.vote ∧ nays = 0) ∨ (¬d.vote ∧ yays = 0) then          ▷ No peer disagrees
16:         return false
17:     end if
18:     d.avalanche_counter ← d.avalanche_counter + 1
19:     next ← AVALANCHE_CUTOFFS[d.avalanche_state].next
20:     if next ≠ d.avalanche_state ∧ d.avalanche_counter ≥ AV_MIN_DUR
                ∧converge_pct ≥ AVALANCHE_CUTOFFS[next].at_pct then
21:         d.avalanche_state ← next
22:         d.avalanche_counter ← 0
23:     end if
24:     if proposing then
25:         θ ← AVALANCHE_CUTOFFS[d.avalanche_state].threshold
26:         total ← |d.votes| + 1
27:         changed ← ( ⌊100 · (yays + d.vote)/total⌋ > θ)
28:     else
29:         changed ← (yays > nays)                                 ▷ Not proposing: follow the majority
30:     end if
31:     if changed = d.vote then
32:         return false
33:     end if
34:     d.vote ← changed
35:     return true
36: end function
```

Proposals. A Proposal is a peer's current transaction set (Algorithm 5). When one arrives, the validator records the peer's position, updates that peer's vote on every dispute, and opens disputes for any new disagreement. A proposal with the sentinel revision VER_LEAVE is a bow-out: the peer's votes are removed from every dispute and the peer is ignored for the rest of the round.

Algorithm 5 Handling a peer's Proposal

```
1:  upon proposal p from a peer
2:      if p.prev_ledger ≠ prev_ledger ∨ p.pk ∈ dead_nodes then
3:          return
4:      end if
5:      if peer_positions[p.pk] ≠ ⊥ ∧ p.ver ≤ peer_positions[p.pk].ver then      ▷ Old proposal
6:          return
7:      end if
8:      if p.ver = VER_LEAVE then                                                ▷ Bow-out
9:          for d ∈ disputes do
10:             d.votes[p.pk] ← ⊥
11:         end for
12:         peer_positions[p.pk] ← ⊥
13:         dead_nodes ← dead_nodes ∪ {p.pk}
14:         return
15:     end if
16:     peer_positions[p.pk] ← p
17:     for d ∈ disputes do
18:         s ← (d.tx ∈ p.txs)
19:         if d.votes[p.pk] ≠ s then                                            ▷ Peer's vote changed
20:             d.votes[p.pk] ← s
21:             peer_unchanged ← 0                                               ▷ vote changed
22:         end if
23:     end for
24:     createDisputes(p.txs)
25: end upon
```

Detecting consensus. haveConsensus ends the Establish (Algorithm 6). It succeeds when at least CONSENSUS_PCT % of the proposers agree on the validator's transaction set or when every dispute has stalled. That is, every dispute is in its final avalanche state for a certain number of ticks, and has a supermajority for or against the transaction. Lastly, if the round has run too long, the validator bows out and accepts its current set, ending the round with a partial validation.

Accepted phase

Finish the round. doAccept closes the round (Algorithm 7). It builds the new ledger from txs on top of prev_ledger and, provided that ledger extends the fully validated tip, signs and broadcasts a Validation for it. The validation is full if the validator was proposing, partial otherwise. Full validations count towards quorum and finality while partial validations only count towards the fork-choice rule.

Finality. On each incoming Validation (Algorithm 8) the validator stores it, adds its ledger to the tree, and records it as the issuer's latest vote. checkAccept then counts the full validations for that ledger: once a ledger collects a quorum of $\left\lfloor \frac{4|\mathsf{UNL}|}{5} \right\rfloor + 1$, the validator sets it as validated_ledger.

4.4 The fork-choice rule

Overview. Due to network delays and/or Byzantine behavior of some validators, the ledgers will not create a single chain, but rather a tree with multiple branches. The fork-choice rule selects one ledger from this tree, the *preferred* ledger, and the validator will try to extend it this round. This choice depends on the validations (partial or full) observed by the validator.

Preferred ledger. getPreferred walks down the tree from genesis (Algorithm 9). At each step it takes the child with the most support, where a ledger's support is the number of validators whose latest validation is for that ledger or for a descendant of it (branchSupport). It descends into the next child

Algorithm 6 Conditions to move on to ACCEPTED phase.

```
1: function haveConsensus(τ)                          ▷ τ: time spent in Establish
2:     agree ← |{p ∈ peer_positions.values() : p.txs = txs}|
3:     if |peer_positions| < ⌊3 · prev_proposers/4⌋ ∧ τ < prev_establish_time + ESTABLISH_WAIT then
4:         return false                                ▷ Participation window for slow peers
5:     end if
6:     if disputes ≠ ∅ ∧ ∀d ∈ disputes : stalled(d) then
7:         return true                                 ▷ Every dispute stalled with a supermajority
8:     end if
9:     total ← |peer_positions|
10:    if proposing then
11:        total ← total + 1
12:        agree ← agree + 1
13:    end if
14:    if ⌊100 · agree/total⌋ ≥ CONSENSUS_PCT then    ▷ Supermajority on our side
15:        return true
16:    end if
17:    ▷ Too much time spent
18:    if τ > min{prev_establish_time · CONSENSUS_FACTOR, CONSENSUS_MAX} then
19:        broadcast(Proposal(prev_ledger, txs, VER_LEAVE, pk))
20:        proposing ← false
21:        return true
22:    end if
23:    return false
24: end function

25: function stalled(d)
26:    current ← AVALANCHE_CUTOFFS[d.avalanche_state]
27:    next ← AVALANCHE_CUTOFFS[current.next]
28:    if next.at_pct > current.at_pct ∨ d.avalanche_counter < AV_MIN_DUR then
29:        return false                                ▷ Not in Stuck, or not there long enough
30:    end if
31:    if peer_unchanged < AV_STALLED_DUR then         ▷ Votes changed recently
32:        return false
33:    end if
34:    yays ← |{vote ∈ d.votes.values() : vote}|
35:    nays ← |{vote ∈ d.votes.values() : ¬vote}|
36:    support ← yays · 100
37:    total ← nays + yays
38:    if proposing then                               ▷ Count our vote if proposing
39:        support ← support + d.vote · 100
40:        total ← total + 1
41:    end if
42:    w ← ⌊support/total⌋
43:    return w > CONSENSUS_PCT ∨ w < (100 − CONSENSUS_PCT)   ▷ Supermajority on one side
44: end function
```

Algorithm 7 Accepting the round

```
1: function doAccept()
2:     phase ← Accepted
3:     L ← (prev_ledger, prev_ledger.seq + 1, txs)
4:     addLedger(L)
5:     ▷ Extends the fully validated chain
6:     if ancestorOf(L, validated_ledger.seq) = validated_ledger then
7:         v ← (L, pk, proposing)                                  ▷ Validation message
8:         broadcast(v)
9:         last_validation[pk] ← v
10:        validations[(pk, L)] ← v
11:        last_issued_seq ← L.seq
12:    end if
13:    checkAccept(L)
14:    prev_proposers ← |peer_positions|                           ▷ For next round's Open phase
15:    prev_establish_time ← now() − establish_start
16:    startRound(L, true)                                         ▷ Start next round
17: end function
```

Algorithm 8 Handling a peer's Validation and finality

```
1: upon validation v from a peer
2:     validations[(v.pk, v.ledger)] ← v
3:     addLedger(v.ledger)                                         ▷ Add to ledger tree
4:     current ← last_validation[v.pk]
5:     if current = ⊥ ∨ v.ledger.seq > current.ledger.seq then
6:         last_validation[v.pk] ← v
7:     end if
8:     checkAccept(v.ledger)
9: end upon

10: function checkAccept(L)
11:    q ← ⌊4|UNL| / 5⌋ + 1                                        ▷ Quorum size
12:    k ← |{((pk_i, L'), v) ∈ validations : L' = L ∧ v.full}|     ▷ Full validations for L
13:    if k ≥ q ∧ (validated_ledger = ⊥ ∨ L.seq > validated_ledger.seq) then
14:        validated_ledger ← L
15:    end if
16: end function
```

as long as its lead over the runner-up exceeds the number of validators that have not yet voted this deep (uncommittedBelow); otherwise the walk stops and returns the current ledger.

Which ledger to extend next. getPrevLedger turns the preferred ledger into the parent for the next round. It jumps to the preferred ledger when that is ahead of, or on a different branch than, the ledger currently being extended, and stays put otherwise. It never drops below the validated tip.

Re-org and bow-out. This check runs on every tick, at the top of Algorithm 2. If getPrevLedger disagrees with the ledger being extended, the validator abandons the round. If it was in Establish, it first broadcasts a bow-out. This is a Proposal with the maximal version number VER_LEAVE, which no later proposal can override. It then restarts on the preferred ledger as a follower (proposing = false). A follower round sends no proposals, follows the majority on every dispute, and ends in a partial validation.

5 Analysis

Based on the protocol description of Section 4, we have identified some points of interest which would help us both to argue about the total security of the protocol and to understand it better. The aim is for the following conjectures to be formalized and proved or disproved in the form of lemmas and theorems under various assumptions and conditions.

First, we turn our attention to the avalanche sub-protocol. This part is of high importance in order for the protocol to reach agreement and finalize ledgers. Otherwise, the protocol will not be able to make progress and transactions will not make it on-chain. Therefore, we are going to investigate the termination of the protocol under various conditions. In particular, we will argue about the protocol terminating when there are no adversarial validators, but honest parties have different inputs, and when there are adversarial validators and the inputs of the honest parties may vary. We expect the protocol to be able to terminate in each case, and we will further examine the output of each honest party and when agreement is guaranteed.

Next, we will investigate the fork-choice rule. Specifically, we would like to prove that an honest party never jumps to a fork that does not extend their fully validated chain, as such a jump could indicate either a safety or a liveness violation. We will also examine how much an adversary can manipulate the accounting of validations in the fork-choice rule, and whether a balancing attack is feasible.

Moreover, we are interested in the liveness guarantees of the protocol. What are the network conditions under which the protocol makes progress? Is there an equivalent property of Chain Growth that we can guarantee for honest parties? How does the protocol recover from the honest parties being partitioned across different forks? All of these questions are of high importance for the security and the performance of the protocol.

Lastly, the UNL overlap and the adversarial assumption form a critical part of the protocol, and have been discussed in previous work with different conclusions (see Section 2). We will examine the protocol under certain assumptions of the UNL overlap and the adversarial resilience and see how the quorum arguments and the liveness guarantees are affected.

Algorithm 9 Ledger tree accounting and the fork choice rule

```
1: function addLedger(L)
2:     while L ≠ ⊥ ∧ L ∉ ledgers do
3:         ledgers ← ledgers ∪ {L}
4:         L ← L.parent
5:     end while
6: end function

7: function branchSupport(L)                    ▷ Validations on L or descendants of L
8:     return |{v ∈ last_validation.values(): ancestorOf(v.ledger, L.seq) = L}|
9: end function

10: function uncommittedBelow(s)        ▷ Number of validators with last validation earlier than s
11:     return |{v ∈ last_validation.values(): v.ledger.seq < s}|
12: end function

13: function getPreferred()
14:     current ← genesis
15:     while true do                                          ▷ Walk down the ledger tree
16:         children ← { L ∈ ledgers: L.parent = current }
17:         if children = ∅ then
18:             break
19:         end if
20:         unc ← uncommittedBelow(max(current.seq + 1, last_issued_seq))
21:         best ← arg max_{L∈children} (branchSupport(L))          ▷ Break ties lexicographically using id
22:         if |children| = 1 then
23:             margin ← branchSupport(best)
24:         else
25:             runnerup ← arg max_{L∈children\{best}} (branchSupport(L))
26:             margin ← branchSupport(best) − branchSupport(runnerup)
27:             margin ← margin + (H(best) > H(runnerup))                ▷ Break ties using id
28:         end if
29:         if unc = 0 ∨ margin > unc then
30:             current ← best
31:         else
32:             break
33:         end if
34:     end while
35:     return current
36: end function

37: function getPrevLedger()
38:     pref ← getPreferred()
39:     if pref.seq > prev_ledger.seq then
40:         return pref
41:     end if
42:     if ancestorOf(prev_ledger, pref.seq) ≠ pref then          ▷ Jump to different branch
43:         return pref
44:     end if
45:     return prev_ledger
46: end function
```

References

[1] David Schwartz, Noah Youngs, and Arthur Britto. The Ripple protocol consensus algorithm. Ripple Labs Inc. White Paper, 2014.

[2] Frederik Armknecht, Ghassan O. Karame, Avikarsha Mandal, Franck Youssef, and Erik Zenner. Ripple: Overview and outlook. In *Trust and Trustworthy Computing (TRUST 2015)*, volume 9229 of *Lecture Notes in Computer Science*, pages 163–180. Springer, 2015.

[3] Brad Chase and Ethan MacBrough. Analysis of the XRP ledger consensus protocol. *arXiv preprint arXiv:1802.07242*, 2018.

[4] Lara Mauri, Stelvio Cimato, and Ernesto Damiani. A formal approach for the analysis of the xrp ledger consensus protocol. In *Proceedings of the 6th International Conference on Information Systems Security and Privacy. 1*, pages 52–63. SciTePress, 2020.

[5] Ignacio Amores-Sesar, Christian Cachin, and Jovana Mićić. Security analysis of ripple consensus. *arXiv preprint arXiv:2011.14816*, 2020.